What survives on a small team's rig: re-testing 22 robot-learning papers with cheap hardware and little data

Ravindu Somawansa
Shisso Robotics
ravindu.somawansa@shissorobotics.com

Abstract. Most robot-learning papers are tested in simulation or on rigs that cost more than $20k; this is the case for 16 of the 22 papers we studied. We re-tested 37 of their claims on a rig of about €1,500, an SO-101 arm with a Jetson Orin NX, with one protocol: the bar is fixed before the test, and the cheapest baseline, a signal we get for free, is run first on the same data. Of the 11 claims we could compare with such a baseline, 5 lost to it; the clearest case is two world models that rank the action chunk the arm really executed worse than the actions alone do. Over all claims, 12 hold and 7 fail with the authors' method on our data; 11 more fail only in our own variant of the method, because of our inference engine, or on a task outside the paper's regime. Many of these failures trace back to our small, short-task data as much as to the methods, and we list for each one what on our side may explain it. The code, the data and every number are public.

Code and rows: `github.com/ravediamond/paper-register-reproducibility` **Reproduce:** `make check`. Every number carries its source file and protocol commit.

1 Introduction

Of the 22 papers in this study, 16 were shown in simulation or on a rig above $20k, and only 2 on a rig under $5k like ours. A small team that wants to use one of these methods has two options: trust the paper, or re-test the claim on its own hardware, which is slow and costly. The question we ask is whether a claim still holds for a small team, with cheap hardware, little data and short tasks.

We re-tested the claims one at a time on a single cheap rig, with the same protocol for each: we fix the bar before the test, and we first run the cheapest baseline, a signal we already have for free, on the same data. The baseline turned out to be the most useful part. LeWorldModel, for instance, predicts the next state of our arm far better than simply copying the last one, but it ranks the chunk the arm really executed worse than the actions alone do. Without the second baseline, we would have called it a success.

Over all the claims, 12 of 37 hold. We split the failures by whose method failed: 7 fail with the authors' method on our data, 8 fail in a variant of the method we had to build ourselves, 2 fail because of our inference engine, and 1 fails on a task outside the regime the paper targets. Section 5 looks at the baselines, Section 6 draws a testing order from the failures, and Section 7 asks, for each failure, how much comes from the method and how much from our data and setup. A failure on our rig is a statement about this rig and this data, not about the method on the hardware it was shown on.

2 Rig and data

The rig is an SO-101 arm (five joints and a gripper) with a top camera and a wrist camera. Inference runs on a Jetson Orin NX 16 GB, and training runs on rented cloud GPUs. The priced parts come to €1,350; with the 2 small parts not priced yet, we put the rig at about €1,500. All the cloud GPU time came to $42.30.

We used four sets of data. The first is our own teleoperated demonstrations (`so101_bowls_all_trimmed`): 182 episodes and 61,462 frames at 30 fps, about 34 minutes in total, on two short tasks where one bowl is placed into another. We split them into 157 training and 25 validation episodes, and used them to train and test the world models and the VLA policies. The second is rollouts of our trained policies on the arm (`so101-bowls-policy-rollouts`): 69 episodes, 44 successes and 25 failures, labelled by the operator, used for the failure monitors. Most monitor tests use a main set of 50 of these episodes (42 successes, 8 failures), which leaves out the episodes of one policy family; FAIL-Detect and SAFE split the 44 successes in half, 22 to calibrate and 22 to test, over 20 random splits. A separate set of 88 of our own episodes, recorded with the top camera only, was used to check the reward model on our rig. The third set is ArmnetBench [14], 2,099 public labelled SO-101 rollouts from 7 policies, used for the reward models and judges. The last is 20 recorded episodes with 254 chunk boundaries, used for the chunking and latency tests.

This data has clear limits. All tasks are short, about 11 s per episode, and none needs several steps or memory, so we say nothing about long-horizon tasks. There is one task family, one arm and one room. The outcome labels come from a single operator and were not given blind.

3 What we tested and how

We chose papers from the parts of the stack our rig runs, and they fall into four kinds (Table 1). World models (7 papers) learn a latent from the camera to predict the next state or to plan; we test whether the latent predicts or

plans better than a signal we get for free. Failure monitors and judges (8 papers) say whether an episode is failing or how far it has progressed; we test whether they catch failures better than a trivial signal. VLA policies (3 papers) follow an instruction on the arm; we test whether they succeed on our arm. Chunking and edge inference (4 papers) concern how a policy's action chunks are produced and joined; we test whether the joins are smoother and how long a chunk takes on the Orin.

Every test follows two rules. First, the bar is written before any result is seen, because a bar written after the result tends to drift toward it. The bars are ours: where the paper reports a number for the claim, the bar is set from it, and the register keeps that number next to ours. Second, before the method itself, we run the cheapest baseline of its kind on the same data. It is built from what we already have for free: the last frame, the joint values we send to the arm, the length of the episode, or the method with its main idea switched off. If the method loses to this baseline, its bar hardly matters.

Each claim gets one of four verdicts: it holds (the bar is met), it fails (the bar is missed), it is not proven (the test ran but cannot decide), or it is not tested yet. A paper fails if any of its claims fails. Because a failure can come from us rather than from the method, we split each failure by whose method failed: the authors' method run on our data, a variant we had to build because a part was not released or did not fit our rig, our inference engine, or a task outside the regime the paper targets. Two of the VLA tests are weaker than the rest: SmolVLA and VLA-JEPA were judged by the operator over 10 trials with no numeric bar fixed before, and 9 successes out of 10 is compatible with a true rate anywhere between 60–98 % (Wilson score interval, 95 %).

Where we could, we also ran a control on our own conclusions. TurboVLA is the clearest case. We believed its architecture made our port of the policy ignore its instruction, and we fixed the bar first: if swapping the task moved the output by $\geq 5°$, the bug was in our port; if by $< 1°$, the problem was the data. With the upstream trainer on our data, swapping the task moved the output by only 0.020°, while their released LIBERO checkpoint, run on our rig, moved it by 5.5°. The problem was our data, not their architecture. We withdrew the claim, and it is not counted in this paper.

4 Results

Table 2 and Figure 1 give the verdicts, and Table 1 breaks them down by kind of paper. Of the 37 claims, 12 hold, 3 are not proven and 4 are not tested yet. Of the 18 that fail, 7 fail with the authors' method on our data, 8 in our own variant, 2 because of our engine and 1 on a task outside the paper's regime. Counted by paper, 4 of the 22 reproduce, and 5 fail with the authors' method. D-JEPA is counted as not tested: we ran it, but only with proxy labels, not with the success labels the paper uses.

The pattern by kind looks sharp, with all 8 failure monitors and judges failing and all 3 VLA policies reproducing, but neither half is strong evidence. Most of the monitor failures are in our own variants, and two of the three VLA results rest on 10 trials with no numeric bar. The appendix gives every paper with its claim, its bar and our result.

5 The cheapest baseline

The cheapest baseline is the signal we get for free on the same data as the method (Table 1). It needs no training and runs in minutes. We compared 11 paper claims with such a baseline (Table 3). The method clearly beat it in 4 cases, beat it by less than its bar in 2, and lost to it in 5. The losses are not equally strong. LeWorldModel and DINO-WM are the clearest, but they are the same test on the same data, run on two models. ContactGuard's loss is in our variant, Robometer's comes from how the dataset ends its episodes (below), and adaptive chunking's 0.973 against 0.940 has no interval. A paired bootstrap over episodes for all 11 rows is still to be run; until then, the count of losses should be read with this in mind.

The choice of baseline decides the verdict, even for a single model. LeWorldModel predicts the next latent far better than copy-last, with an error of 0.31 against 0.89, and that claim holds. Yet the same model ranks the chunk that was really executed worse than the actions alone do, 0.60 against 0.68–0.90, and that claim fails. Predicting well is not the same as planning well, and testing only one of the two would give the wrong picture.

The same baseline is also not equally strong on all data. On ArmnetBench, episode length beats the reward model (0.938 against 0.818), because there a failed episode tends to run until the time limit, so the length gives the label away. On our own 88 episodes, where failures do not run to the time limit, length is close to chance (0.521). What a judge must beat therefore depends on who stopped the episode, and the baseline has to be computed on one's own data.

Copy-last and the joints are strong partly because our task is short and simple: little moves apart from the arm, and the arm's state is already in the joints. On long tasks with many moving objects a learned latent has more to add, and we did not test that. Some papers report a cheap baseline themselves (shuffled actions for ReAP, a random pick for PACT-VLA, net joint displacement for HWM), and on our data those methods beat it too; Robometer reports no episode-length baseline. In practice, the baseline is the first thing to run: if the method loses to it, one can stop there, and only if it wins is the bar worth checking.

Table 1: The four kinds of papers, what we test in each, the cheapest baseline we build for it, and the verdicts.

kind of paper	papers	what we test	cheapest baseline	verdicts
world models	7	predict the next state; rank or plan actions	copy-last (next state = current state); the commanded joints or the actions alone	1 reproduce, 4 fail, 1 not proven, 1 not tested
failure monitors and judges	8	catch failures; track progress	episode length; the same model with shuffled actions	8 fail
VLA policies	3	succeed on our arm	none: success over 10 trials judged by the operator, no numeric bar fixed before	3 reproduce
chunking and edge inference	4	smoother joins between chunks; time per chunk	the same policy without the trick; the real-time budget on the Orin	2 fail, 2 not proven



Figure 1: the 37 paper claims by verdict: 12 reproduce, 7 fail with the authors' method on our data, 11 fail only in our variant of the method, because of our inference engine or outside the paper's regime, and the rest are open.

Table 2: Verdicts by paper and by claim. A paper fails if any of its claims fails; failures are split by whose method failed.

verdict	papers	claims
reproduces	4	12
fails, authors' method	5	7
fails, our variant	7	8
fails, our engine	1	2
fails, outside the paper's regime	1	1
not proven	3	3
not tested	1	4

6 A testing order

The failures point to three checks, in this order. First, time the method on the target device: adaptive chunking needs 4,751 ms for one decision against a budget of 298 ms, partly because our engine samples without batching, and this is the cheapest failure to find. Second, compare it with the cheapest baseline on one's own data: this is where the world models fail, since the commanded joints already hold the arm's state on a short task, and it is also where a dataset can mislead, as episode length does on ArmnetBench. Only then is it worth checking the method against its bar. Of the three, only the learned-latent pattern rests on more than one case, so we offer this as a working order, not as a taxonomy of how methods fail.

7 Why they failed here: the method, our data, or our setup

For each failure, Table 4 says whose method failed and what on our side may explain it. Most failures have at least one such cause, so a failure here is a failure on this rig, with this data, and not a verdict on the method in general.

Our data is small and simple. We have 157 training episodes of one short task, about 11 s each, where the papers use hours of varied data; HWM, for instance, uses about 130 h against our 0.6 h. On a short task with one arm, the commanded joints already hold most of the state, which leaves little for a latent to add. The task is also easy for our policy, since SmolVLA is at its ceiling on it, so there is little uncertainty for an entropy rule to read: the chunk length chosen by adaptive chunking sat at its minimum in 86 % of decisions. For H-JEPA, our task lies outside the regime the paper targets, as there is no gap between slow and fast parts of the scene, so we count its failure as out of scope rather than a failure of the method.

We also have too few failures. Our policies mostly succeed, and the main rollout set has only 8 failures in 50 episodes. A bar such as five failures caught out of eight is coarse, and a single episode can change the verdict (ReAP, VLA-Corrector, SURVEYOR).

In 7 papers our version of the method is not exactly theirs. For ContactGuard we used mixed policies and two cameras, where the paper uses one policy and three cameras. For FAIL-Detect and SAFE we used our world model's surprise as the score instead of their own scores. ReAP's risk head ran on our world model's latent, and our VLA-Corrector verifier works on pooled image statistics instead of their 54 features with the arm state. The drafter of SURVEYOR [26] is not released, and REMORA's progress atoms are written by hand. These are failures of our variants, and we do not claim that the authors' methods fail.

Our inference engine matters as well. vla.cpp samples sequentially and has no batching, so adaptive chunking

Table 3: The paper claims we compared with a cheap baseline, on the same data.

method	baseline (what is measured)	method	baseline	outcome
LeWorldModel	copy-last (latent error at 30 steps, lower is better)	0.31	0.89	beat
LeWorldModel	actions alone (executed chunk ranked first)	0.60	0.68–0.90	lost
DINO-WM	actions alone (executed chunk ranked first)	0.61	0.68–0.90	lost
RoboJEPA	copy-last (9-step latent error)	11.8 % below copy-last	–	beat, under its bar of 15 %
ContactGuard	the current latent (failure AUROC)	0.70	0.82	lost
ReAP	shuffled actions (failure AUROC)	0.789	0.708	beat
Robometer	episode length (failure AUROC, ArmnetBench)	0.818	0.938	lost
HWM	net joint displacement (cosine to the macro-action)	0.608	0.551	beat
Adaptive chunking	fixed chunk length (boundary jump, lower is better)	0.973	0.940	lost (no interval)
PACT-VLA	a random pick (error of the executed window)	$-0.27°$ against random	–	beat
REMORA	per-frame readout (readout flips)	0.70 × per-frame	–	beat, under its bar of 0.5 ×

pays for 20 full passes, while the paper's 106 ms was measured with batching on an A800. For PACT-VLA, the time of the check is projected rather than measured. Finally, several tests were offline only: planning ranked recorded chunks, and chunking measured smoothness on recorded episodes. None of these ran in closed loop on the arm.

8 Limitations

Our data covers one short task family, about 11 s per episode, with one arm, two cameras and about 34 minutes of demonstrations, so it says nothing about long-horizon tasks or varied scenes. There are few failures to test the monitors on, 8 in the main set of 50. Many methods are our own variants rather than the authors' code end to end (Table 4), and the planning and chunking tests ran offline, never in closed loop on the arm. A single operator ran the arm and labelled the outcomes without being blind to them, many tests use small samples and a single training seed, and few results come with a confidence interval; a bootstrap over episodes for the baseline comparisons is still to be run. Two VLA results have no numeric bar and rest on 10 trials. The bars are ours, not the papers'. We chose the papers from the stack we run, so the rates describe these papers and not the field. Finally, our own mistakes are possible, as the TurboVLA case shows (Section 3).

9 Related work

Two works evaluate policies on the same arm: a real-world VLA benchmark with a taxonomy of failures and recoveries [13], and ArmnetBench, a fleet of SO-101 cells that evaluates policies at scale [14]. In simulation, vla-eval runs VLAs across 14 benchmarks [8], and another benchmark studies the quality of demonstrations [21]. These works rank policies. We instead take the claim a paper makes about its method and test it against a bar fixed in advance, on one cheap rig. The closest work is RoboSeed [27], which scores VLA and world-action models on five dimensions, including reproduction evidence and deployment affordability; its authors note that the score is not a probability that a result is correct. We give one verdict per claim on one rig, with the data behind it and the reason we think it failed.

10 Conclusion

We re-tested 37 claims from 22 robot-learning papers on a rig of about €1,500, with under $50 of cloud GPU. 12 held, and 7 failed with the authors' method on our data. The most useful part was the cheapest baseline: the same world model could pass one baseline and fail another, and 5 of the 11 methods we compared with one lost to a signal we had for free, although only two of these losses are clear so far. Our advice to anyone re-testing a paper on a cheap rig is to run that baseline first, on their own data, with the bar fixed before the test. Everything is public, and every number can be rebuilt from the repository. Authors who think a failure here is ours (Table 4) are welcome to tell us, and we will correct it.

References

[1] DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning, 2024. arXiv:2411.04983.

[2] Can We Detect Failures Without Failure Data? Uncertainty-Aware Runtime Failure Detection for Imitation Learning Policies, 2025. arXiv:2503.08558.

Table 4: Each failed or not-proven paper, whose method failed, what we measured, and what on our side may explain it.

failed paper	whose method	our result	what may be on our side
LeWorldModel	authors'	planning rank 0.60, action-only 0.68–0.90	one short task, 157 episodes, one seed; offline ranking only
DINO-WM	authors'	planning rank 0.61, action-only 0.68–0.90	same data as LeWorldModel; one training
RoboJEPA	authors'	11.8 % below copy-last, bar 15 %	near miss; the 1B model reached 15.8 % on 30 clips only
H-JEPA	outside its regime	progress ρ −0.73, bar −0.80; false stalls 52 %	our task is outside the paper's regime
HWM	not proven	not proven: high level worse at 2 s	0.6 h of one task against their 130 h
ContactGuard	our variant	AUROC 0.70, current latent 0.82	mixed policies and 2 cameras against one policy and 3 cameras
FAIL-Detect and SAFE	our variant	22.7 % of successes flagged at α 5 %	our score, not theirs; 22 successes for calibration
ReAP	our variant	3 of 8 failures caught	their risk head on our world model's latent, not their features; 8 failures
VLA-Corrector	our variant	2 of 8, AUROC 0.59	our features, not their 54; 8 failures
Robometer	authors'	AUROC 0.818, length 0.938	zero-shot only; ArmnetBench failures run to the time limit (on our rollouts length is near chance)
SURVEYOR	our variant	threshold 1.249 against 0.20; 3 of 8	their drafter not released; 8 failures
REMORA	our variant	readout flips 0.70 of the per-frame readout	hand-written progress atoms
Adaptive chunking	authors' (latency: our engine)	4,751 ms per decision; length 2 in transport and at the grasp	sequential engine, no batching; task at ceiling for our policy
PACT-VLA	our engine	37 ticks of delay against 8	check time projected; sequential engine
Real-Time Chunking	not proven	not proven: 71–73 % smoother, no arm run	offline smoothness only
Horizon × depth	not proven	not proven: 0.78, bar 0.70	one policy, one task; arm test not run

[3] SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics, 2025. arXiv:2506.01844.

[4] Real-Time Execution of Action Chunking Flow Policies, 2025. arXiv:2506.07339.

[5] SAFE: Multitask Failure Detection for Vision-Language-Action Models, 2025. arXiv:2506.09937.

[6] VLA-JEPA: Enhancing Vision-Language-Action Model with Latent World Model, 2026. arXiv:2602.10098.

[7] Robometer: Scaling General-Purpose Robotic Reward Models via Trajectory Comparisons, 2026. arXiv:2603.02115.

[8] vla-eval: A Unified Evaluation Harness for Vision-Language-Action Models, 2026. arXiv:2603.13966.

[9] LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels, 2026. arXiv:2603.19312.

[10] Hierarchical Planning with Latent World Models, 2026. arXiv:2604.03208.

[11] Adaptive Action Chunking at Inference-time for Vision-Language-Action Models, 2026. arXiv:2604.04161.

[12] When Does LeJEPA Learn a World Model?, 2026. arXiv:2605.26379.

[13] Benchmarking Vision-Language-Action Models on SO-101: Failure and Recovery Analysis, 2026. arXiv:2606.08881.

[14] ArmnetBench v0.1: Parallel Real-World Evaluation of Manipulation Policies on a Low-Cost Arm Farm, 2026. arXiv:2607.24481.

[15] ContactGuard: Pre-Contact Execution Monitoring with Action-Conditioned Latent World Models, 2026. arXiv:2608.13438.

[16] VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies, 2026. arXiv:2609.06508.

[17] D-JEPA: A Decision-Aligned Latent World Model, 2026. arXiv:2609.24749.

[18] H-JEPA: End-to-End Learning of Hierarchical World Models for Visual Planning, 2026. arXiv:2610.06805.

[19] RoboJEPA: Scaling Robotic Latent World Models, 2026. arXiv:2610.10515.

[20] ReAP: From Visual Failure Recognition to Action-Conditioned Risk Prediction for Runtime Failure Monitoring, 2026. arXiv:2610.16309.

[21] What Matters in Quality-Aware VLA Pretraining: A Benchmark and Empirical Study, 2026. arXiv:2610.18869.

[22] REKI: Registering Knowledge in VLAs, 2026. arXiv:2610.22178.

[23] PACT-VLA: Speculative Action-Chunk Verification with Pre-Commit Observations, 2026. arXiv:2610.28019.

[24] Coupling Execution Horizon and Sampling Depth in Flow-Matching VLA Policies, 2026. arXiv:2610.32102.

[25] Tracking Task Progress Through Errors and Recovery:A Benchmark and Recurrent Distributional Model, 2026. arXiv:2610.39125.

[26] SURVEYOR: Certified Speculative Plan Consumption for Latent World-Model Planning. NeurIPS 2026 workshop, 2026. neurips.cc/virtual/2026/175857.

[27] Xin Ma et al. RoboSeed: Multidimensional Evaluation of Vision-Language-Action and World-Action Models: Capability, Resource Affordability, and Research Evidence, 2026. arXiv:2610.40026.

Appendix: every paper

paper	their rig	claim tested	bar	our result	verdict
LeWorldModel [9]	sim	A small JEPA world model learns the arm's dynamics well enough to plan with it.	Dynamics: beat copy-last, every corruption ≥ 1.3 at $K = 30$. Planning: rank the executed chunk first among 8 in ≥ 0.8 of anchors.	dynamics 0.31 vs 0.89 copy-last (reproduces); planning top-1 0.60 vs 0.68–0.90 for an action-only baseline (fail)	fail (authors' method)
DINO-WM [1]	sim	A predictor on frozen DINOv2 patches plans better than one on a learned latent.	G1b top-1 ≥ 0.8 at some target distance.	arm-pose R^2 0.92 (vs 0.57 for W1); G1b top-1 0.55–0.61; no-model baseline 0.68–0.90	fail (authors' method)
RoboJEPA [19]	>$20k	Pretrained RoboJEPA predicts our latents, uses the actions, and runs on the robot's computer.	9-step L1 ≥ 15 % below copy-last (300M) and ≥ 5 % below shuffled actions; 2-view encoder ≤ 200 ms on the Orin.	11.8 % below copy-last (300M; 1B 15.8 % on 30 clips); 9–10 % below shuffled; encoder 526 ms on the Orin NX	fail (authors' method)
H-JEPA [18]	sim	A slow second level makes latent distance track task progress.	Progress monotonicity Spearman $\rho \leq -0.80$; false stalls ≤ 10 %.	level-2 ρ -0.73; false stalls 52 %	fail (outside the paper's regime)
D-JEPA [17]	<$5k	A decision-aligned ranking operator on a frozen world model picks better action chunks.	G1b regret ≤ 0.10 and $\leq$ the action-only baseline + 0.05.	regret 0.263 (W1 alone 0.263; action-only -0.007)	not tested (proxy labels only)
ContactGuard [15]	$5–20k	An event-anchored probe on imagined latents predicts grasp failure better than the current latent.	AUROC ≥ 0.85 and ≥ 0.05 above the current latent.	0.70 imagined vs 0.82 current	fail (our variant)
FAIL-Detect and SAFE [2] [5]	>$20k	A conformal band on a learned failure score keeps false alarms at α while catching failures.	Successes flagged $\leq 1/20$; failures flagged ≥ 50 %.	22.7 % of successes flagged at $\alpha = 5$ %; 46 % of failures caught	fail (our variant)
ReAP [20]	$5–20k	A risk head on an action-conditioned latent warns before failures.	$\geq 5/8$ failures caught at $\leq 1/20$ false interventions; action-conditioned iff true $\geq$ shuffled + 0.05.	v1 AUROC 0.93 came from time and length; v2 0.789 vs 0.708 shuffled, 3/8 at 5/42	fail (our variant)
VLA-Corrector [16]	sim	A verifier on observable history flags a failing rollout early enough to correct it.	$\geq 5/8$ failures at $\leq 1/20$ false interventions.	the paper's verifier family on our rollouts (a GRU on pooled RGB statistics and actions): 2/8 caught at 3/42 false interventions, AUROC 0.59; cosine drift of the W1 latent 0/8, AUROC 0.30	fail (our variant)
Robometer [7]	>$20k	A 4B reward model judges SO-101 rollouts zero-shot.	Pooled AUROC ≥ 0.85 and within-policy ≥ 0.8 on ≥ 5 of 7 policies.	pooled AUROC 0.818 on 2099 ArmnetBench rollouts; episode length alone 0.938	fail (authors' method)
SmolVLA [3]	<$5k	A 450M VLA succeeds on an SO-101 task and can run on low-cost hardware.	10-trial operator-judged gate (no numeric bar); chunk inference within the async budget (667 ms).	9/10 on bowls; 974 ms in PyTorch, 238 ms with vla.cpp on the Orin NX	reproduces

Table 5: One row per paper with a claim of its own; FAIL-Detect and SAFE share a row. The bars are ours, written before each test, from the paper's own number where it gives one (that number is kept in the register). Internal names: W1 is our LeWorldModel and W2 our DINO-WM, both trained on our teleoperated data; names such as G1b, H0 or P1 are the ids of our tests in the repository. The rig price of €1,350 counts the parts with a stated price; 2 small parts are not priced yet. Read the verdict against the rig each paper was shown on.

paper	their rig	claim tested	bar	our result	verdict
VLA-JEPA [6]	sim	VLA-JEPA trains cheaply on SO-101 data and runs on the robot.	10-trial operator-judged gate (no numeric bar).	9/10 for $9.7; 355 ms per chunk on the Orin NX	reproduces
REKI [22]	sim	Policies without memory oscillate between poses at a measurable rate.	$\geq$ 10 % of failed episodes oscillate (their criterion).	20.3 % of all episodes, 24.0 % of failures; REKI trained on our data: 19/20 + 19/20 on the arm, oscillates in 1/40	reproduces
Real-Time Chunking [4]	unstated	Conditioning a new chunk on the executing one removes the jump at chunk boundaries.	Best gradient-free method cuts the median jump $\geq$ 50 % against naive async, drift $\leq$ +10 %, within 667 ms.	prefix overwrite cuts the jump 71–73 % against naive async at +0.4 ms; tuned LeRobot identity mode is close	not proven
HWM (hierarchical latent world models) [10]	>$20k	A high-level world model over learned macro-actions beats the low-level model beyond about 1 s (their Fig. 6 crossover).	Crossover in $[1.0, 2.0]$ s; at 2 s high-level error $\leq$ 0.95 $\times$ low-level; learned macro-actions $\geq$ net joint displacement on cosine.	no crossover: W1 $0.382 \pm 0.005 \rightarrow 0.563 \pm 0.007$, high level $0.769 \pm 0.009 \rightarrow 0.759 \pm 0.009$ (0.73–2 s); cosine 0.608 ± 0.007 vs 0.551 ± 0.006	not proven
LeJEPA identifiability [12]	sim	A SIGReg JEPA trained on goal-directed data recovers the true latent state only partially and unevenly.	W1's mean R^2 from latent to joints < 0.80; anisotropy (max $-$ min per-joint R^2) ≥ 0.20; the higher-R^2 model has the higher G1b top-1.	W1 mean R^2 0.565, anisotropy 0.639 (shoulder lift 0.91, wrist flex 0.27); frozen DINOv2-S 0.867 / 0.301	reproduces
SURVEYOR [26]	sim	A threshold on the imagined-vs-reached latent distance accepts good plans and rejects bad ones ($\tau = 0.20$).	τ below the 10th percentile of non-equivalent pairs and $\leq$ 0.40; as a monitor $\geq$ 5/8 failures at $\leq$ 2/42; $\geq$ 0.05 AUROC over a shuffled drafter.	$\tau = 1.249$ on W1 latents of 69 real rollouts; as a monitor 3/8 at 6/42 (S = 30), AUROC 0.756 vs shuffled 0.560	fail (our variant)
Adaptive Action Chunking (AAC) [11]	>$20k	Chunk length chosen from the entropy of N sampled chunks follows the task's phases (long in transport, short at the grasp), lowers the boundary jump, and costs little with batched sampling.	median h* in transport $\geq$ grasp window + 2; jump with AAC + overwrite $\leq$ fixed + overwrite at d = 8; an N = 20 decision $\leq$ 298 ms on the Orin.	h* 2 in transport and 2 at the grasp (86 % of decisions at the floor); jump 0.973 vs 0.940; N = 20 decision 4,751 ms (sequential) on the Orin	fail (authors' method); the latency claim: our engine (no batched sampling)
PACT-VLA [23]	$5–20k	Picking among N sampled chunks by their flow residual under the pre-commit observation fits what happens next better than a random or stale pick, with no added control-step delay.	PACT beats a random pick and the stale-observation residual at d = 8 (CIs < 0); its delay on the Orin $\leq$ the naive delay; at that delay it still beats a single sample.	offline $-0.27°$ vs random and vs the stale residual (both CIs < 0); on the Orin 1,200 ms per cycle, 37 control ticks vs 8	fail (our engine)
Horizon $\times$ depth (Euler steps vs execution horizon) [24]	>$20k	With long open-loop commitment, cutting a flow-matching VLA's Euler steps from 10 to 2 contracts the action cloud around an unchanged centroid, saves most of the inference time and avoids edge-GPU thermal throttling.	median R(2)/R(10) $\leq$ 0.70 with the centroid within 0.05 relative and 3°; N = 2 latency $\leq 0.85 \times$ N = 10 on the Orin, N = 10 within 5 % of 238 ms.	R(2)/R(10) 0.78 (CI 0.73–0.81), centroid 0.018 and 2.35°; 196.1 vs 237.5 ms per chunk (0.83, vision 151 ms); no throttling at N = 10 on the Orin, at MAXN_SUPER or 25 W	not proven
REMORA [25]	>$20k	Reading a progress distribution out at the supported peak nearest the previous estimate steadies the estimate and lets it fall during real setbacks, with no worse detection of the stages.	Against per-frame argmax: premature not higher and missed $\leq$ +2 points (P1); flips halved, CI upper $\leq$ 0.7 (P2); a decrease inside $\geq$ 50 % of the reversal windows with $\leq$ 20 % false regressions (P3).	premature 0.22 vs 0.25, missed equal (P1 pass); flips 0.70 $\times$ argmax, CI 0.48–0.94 (P2 fail); setback recall 2/7, false 0.52 (P3 fail); MAE 0.184 vs median 0.218	fail (our variant)

Table 5, continued.